

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I M. Pranay (192525382)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Pranay

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:

- A. Process live video streams in real time (<30 FPS latency)
- B. Work under varying lighting conditions
- C. Use limited memory and CPU resources
- D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System

A security firm needs a video surveillance system that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents

A digitization company needs an OCR system that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:

- A. Limited processing power (mobile CPU)
- B. Real-time performance
- C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

ANSWER

1. Real-Time Face Recognition System – Edge Deployment

Problem Understanding

The system must recognize faces using a live camera on a low-power device such as Raspberry Pi. It must work in real time, under different lighting conditions, with limited CPU and memory.

OpenCV Pipeline

1. Image Acquisition

- Capture live video using `cv2.VideoCapture()`.
- Resize frames to a smaller resolution to reduce processing time.
- Process selected frames when necessary to maintain real-time performance.

2. Preprocessing

- Convert the frame from BGR to grayscale using `cv2.cvtColor()`.
- Apply histogram equalization using `cv2.equalizeHist()` to improve performance under varying lighting.
- Resize the image to reduce CPU and memory usage.

3. Feature Detection

- Use OpenCV Haar Cascade or LBPH-compatible face detection.
- Detect the face region using `detectMultiScale`.
- Extract only the detected face region for recognition.

4. Face Recognition

- Use **LBPH (Local Binary Pattern Histogram)** because it is lightweight and suitable for low-power devices.
- Train the recognizer using the predefined face dataset.
- Compare the detected face with stored face patterns.
- Display the person's identity and confidence value.

5. Output

- Draw a rectangle around the detected face using `cv2.rectangle()`.
- Display the recognized person's name using `cv2.putText()`.
- Show the processed video using `cv2.imshow()`.

How Constraints Are Satisfied

Constraint	Solution
Real-time processing	Resize frames and process efficiently
Varying lighting	Grayscale + histogram equalization
Limited CPU	Lightweight Haar Cascade + LBPH
Limited memory	Small image resolution and limited stored features
Predefined dataset	Train LBPH using known faces

Conclusion:

A **Haar Cascade + LBPH** pipeline provides a lightweight and feasible solution for real-time face recognition on Raspberry Pi.

2. Automated Video Surveillance with Alert System

Problem Understanding

The system should continuously monitor CCTV footage, detect motion or unusual activity, generate real-time alerts, and minimize false detections caused by noise and moving backgrounds.

OpenCV Pipeline

1. Video Acquisition

- Read CCTV video using `cv2.VideoCapture()`.
- Process frames continuously.
- Resize frames to reduce computational load.

2. Preprocessing

- Convert frames to grayscale.
- Apply Gaussian blur using `cv2.GaussianBlur()` to remove noise.
- Normalize the input frames when required.

3. Motion Detection

- Compare consecutive frames or use background subtraction.
- OpenCV functions such as `cv2.createBackgroundSubtractorMOG2()` can be used.
- Generate a foreground mask containing moving objects.

4. Noise Removal

- Apply morphological operations such as opening and closing.
- Remove very small contours.
- Use area thresholds to ignore insignificant movements.

5. Activity Analysis

- Find contours using `cv2.findContours()`.
- Calculate bounding boxes using `cv2.boundingRect()`.
- Identify significant movement based on object size and movement duration.

6. Alert Generation

- If unusual motion exceeds the predefined threshold, generate an alert.
- Display an alert message on the screen.
- A sound or external notification can also be triggered.

Handling Constraints

- **Dynamic background:** Background subtraction with thresholding reduces unwanted detections.
- **Noise:** Gaussian blur and morphological operations reduce noise.
- **False positives:** Minimum contour-area and motion-duration thresholds are used.
- **Real time:** Frame resizing and efficient OpenCV operations reduce processing time.

Conclusion:

The combination of **background subtraction, contour detection, filtering, and threshold-based alerts** provides an efficient surveillance system with reduced false positives.

3. OCR System for Low-Quality Documents

Problem Understanding

The OCR system must extract text from noisy, low-resolution documents while handling different fonts and illumination conditions and efficiently processing multiple images.

OpenCV Pipeline

1. Image Acquisition

- Load scanned images using `cv2.imread()`.
- Process images individually or in batches.

2. Image Enhancement

- Convert the image to grayscale using `cv2.cvtColor()`.
- Apply Gaussian or median filtering to remove noise.
- Improve contrast using histogram equalization.

3. Illumination Correction

- Normalize uneven lighting.
- Use adaptive thresholding for documents with varying brightness.

4. Image Segmentation

- Apply thresholding using `cv2.threshold()` or `cv2.adaptiveThreshold()`.
- Convert the document into a suitable binary image.
- Use morphological operations to remove unwanted noise.

5. Text Region Detection

- Detect connected components or contours.
- Identify regions containing text.
- Crop text regions when required.

6. OCR

- Send the processed image to an OCR engine such as Tesseract.
- Extract characters and words from the cleaned image.

7. Output

- Store the extracted text.
- Display or save the recognized text for further processing.

How Each Stage Helps

Stage	Purpose
Grayscale	Simplifies image processing
Noise filtering	Removes scanning noise
Histogram equalization	Improves contrast
Adaptive thresholding	Handles uneven illumination
Morphology	Removes small unwanted regions
Segmentation	Separates text from background
OCR	Converts image text into editable text

Conclusion:

A preprocessing pipeline using **grayscale conversion, noise removal, contrast enhancement, adaptive thresholding, and segmentation** improves OCR performance on low-quality documents.

4. Facial Expression Recognition for Mobile Application

Problem Understanding

The application must recognize **happy, sad, and neutral** expressions on a mobile device while maintaining real-time performance and handling slight head movements and lighting changes.

OpenCV Pipeline

1. Image Acquisition

- Capture live frames using the mobile camera.
- Reduce frame resolution to lower CPU usage.

2. Preprocessing

- Convert frames to grayscale.
- Resize the frame.
- Apply histogram equalization to improve lighting robustness.

3. Face Detection

- Use OpenCV Haar Cascade or another lightweight face detector.
- Detect the face using detect Multi Scale.

4. Face Alignment

- Crop the detected face.
- Normalize the face size.
- Handle small movements by detecting the face in each frame.

5. Expression Analysis

- Extract facial features such as eyes, eyebrows, mouth, and facial contours.
- Use a lightweight classifier to classify:
 - **Happy**
 - **Sad**
 - **Neutral**

6. Output

- Draw a bounding box around the face.
- Display the predicted expression using `cv2.putText()`.

Balancing Accuracy and Efficiency

- Use a lightweight face detector.
- Reduce input resolution.
- Process only the detected face region.
- Use grayscale instead of full-color images.
- Apply simple preprocessing for lighting changes.
- Avoid processing every pixel at maximum resolution.

Conclusion:

A lightweight **OpenCV face detection + feature extraction + expression classification** approach provides suitable accuracy while maintaining real-time performance on mobile CPUs.

5. Smart Traffic Monitoring System

Problem Understanding

The system must detect vehicles from live video, track their movement, count vehicles, display statistics, and operate continuously with minimum delay.

OpenCV Pipeline

1. Video Acquisition

- Capture live traffic video using `cv2.VideoCapture()`.
- Read frames continuously.
- Resize frames for faster processing.

2. Preprocessing

- Convert frames to grayscale if required.
- Apply Gaussian blur to reduce noise.
- Use background subtraction for fixed CCTV cameras.

3. Vehicle Detection

- Apply background subtraction using `cv2.createBackgroundSubtractorMOG2()`.
- Detect foreground objects.
- Find contours using `cv2.findContours()`.

4. Vehicle Filtering

- Calculate contour area.
- Remove objects smaller than the predefined threshold.
- Use bounding rectangles with `cv2.boundingRect()`.

5. Vehicle Tracking

- Assign an identifier to detected vehicles.
- Track their center points across consecutive frames.
- Compare the current and previous positions.

6. Vehicle Counting

- Define a virtual counting line.
- When a vehicle crosses the line, increase the vehicle count.
- Maintain separate counts if required for different directions.

7. Visualization

- Draw bounding boxes using `cv2.rectangle()`.
- Draw the counting line using `cv2.line()`.
- Display vehicle count using `cv2.putText()`.
- Show the processed video using `cv2.imshow()`.

System Workflow

Live CCTV → Frame Capture → Preprocessing → Vehicle Detection → Contour Filtering → Tracking → Line Crossing → Vehicle Counting → Statistics Display

How Constraints Are Satisfied

Constraint	Solution
Vehicle detection	Background subtraction + contours
Movement tracking	Centroid/position tracking
Vehicle counting	Virtual line crossing
Continuous operation	Efficient frame processing
Minimum delay	Frame resizing and lightweight operations
Visualization	Rectangle, line and text drawing

Conclusion:

An OpenCV-based pipeline combining **video processing, background subtraction, contour detection, tracking, and line-crossing counting** provides an efficient solution for real-time smart traffic monitoring.